ER Diagram for Electronic Voting System

Module Title and Code: Introduction to Databases, UFCFTK-30-1

Submitted By: Raunak Kumar Singh

Student Number: 24071246

Orbund Id: 11281

Submitted To: Rabin Gautam

TASK 1 (30%)

Construct an ER Diagram for an Electronic Voting System

Design an ER diagram for an electronic voting system to replace the manual voting system in Nepal. Make the following assumptions:

1. **Voter Registration:** Each voter has personal details (name, citizenship number, address).
2. **Election Details:** The system supports multiple elections, each with a unique ID and election date.
3. **Candidates:** Each election has multiple candidates, with details such as name and political party.
4. **Voting Process:** Each voter can vote once per election.
5. **Results:** The system stores the election results, including votes for each candidate.
6. **Security:** Ensure vote confidentiality and prevent tampering.

Requirements:

- **Entities:** Identify entities like Voter, Election, Candidate, Vote, etc.
- **Relationships:** Define relationships (e.g., Voter casts Vote, Election has Candidates).
- **Cardinalities:** Specify cardinalities (one-to-many, many-to-many).
- **Constraints:** Include constraints like voter eligibility (age, citizenship) and one vote per voter.

Deliverables:

1. **ER Diagram:** Include entities, attributes, relationships, and cardinalities. You can use any tools for creating the ER diagram (e.g., draw.io, Lucidchart, Microsoft Visio).
2. **Relations:** Convert the ER diagram into a relational schema with primary and foreign keys.

Introduction

This task involves designing an **ER diagram** for an **Electronic Voting System**. The systems main motive is to remove the traditional voting system of **Nepal** with new and most effective **Electronic Voting System**.

The ER model represents key entities such as **voters**, **elections**, **candidates**, **votes**, and **results**, along their **relationships** and **constraints**.

In addition to the voting process, **voter registration** is also included to better reflect a real-world election system.

Entity-Relationship (ER) Diagram

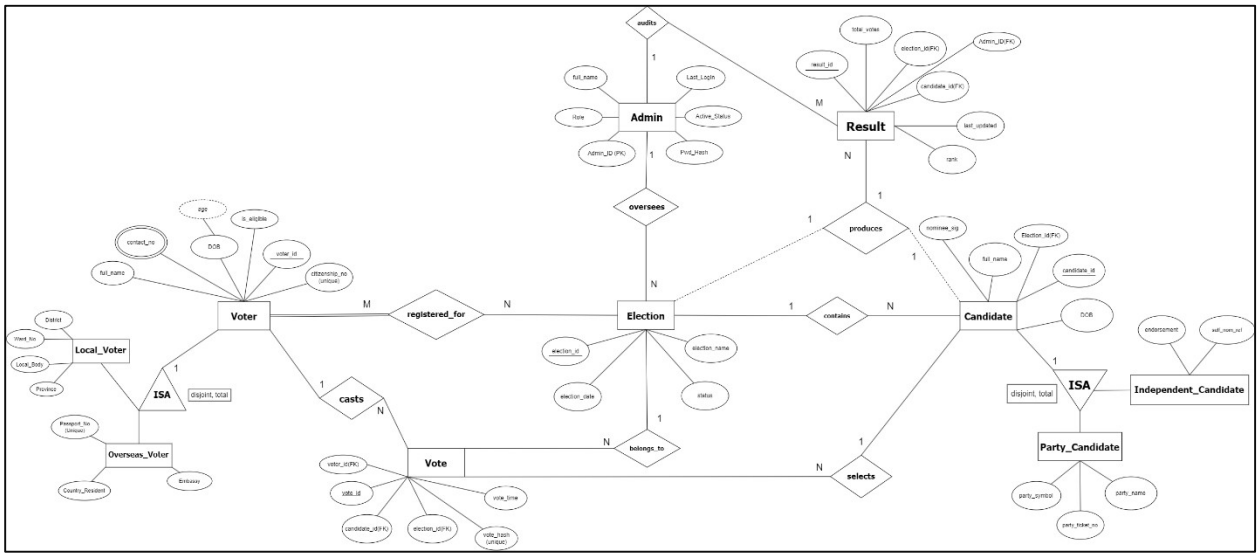

Fig: **ER Diagram for Electronic Voting System** (Created Using **draw.io**)

Entity Description

My **Electronic Voting System** consists of the following entities:

1.**Voter**: Stores information about individuals eligible to vote, including **voter_id**, **full_name**, **citizenship_no**, **DOB**, and **contact_no**.

2. **Local_Voter:** A specialization of **Voter** representing voters residing within the country, including attributes such as **district**, **ward_no**, **local_body**, and **province**.

3. **Overseas_Voter:** A specialization of **Voter** representing voters residing in another nation, including attributes such as **passport_no**, **embassy**, and **country_resident**.

4. **Election:** Represents elections conducted in the system designed by me, which includes **election_id**, **election_name**, **election_date**, and **status**.

5. **Candidate:** Represents individuals participating in elections. Each candidate is associated with a specific election and may either be a **party** or **independent** candidate.

6. **Party_Candidate:** A subclass of **Candidate** containing attributes such as **party_name**, **party_symbol**, and **party_ticket_no**.

7. **Independent_Candidate:** A subclass of **Candidate** representing candidates without party affiliation.

8. **Vote:** Represents each vote cast in the system, including **vote_id**, **voter_id**, **election_id**, **candidate_id**, **vote_time**, and **vote_hash**.

9. **Result:** Represents the outcome of elections, storing **total_votes** for each candidate along with **election_id, admin_id,** and **candidate_id**.

10. **Admin:** Represents system administrators responsible for managing **elections** and auditing the **results**.

Relationships

The relationship between entities is implemented as follows:

1. A voter registers for an election through the **registered_for** relationship, showcasing participation eligibility before voting.
2. A voter **casts** a vote.
3. A vote **belongs to** an election.
4. A vote **selects** a candidate.
5. An election **contains** multiple candidates.
6. An election **produces** results.
7. A candidate **produces** results.
8. An admin **oversees** elections.
9. An admin **audits** results.

Cardinalities

The cardinalities of the relationship in my system are as follows:

1. A voter can register for multiple elections and each election can have multiple voters. **(M: N)**
2. One voter can cast multiple votes. **(1: N)**
3. Each vote belongs to one election. **(N: 1)**
4. Each vote selects one candidate. **(N: 1)**
5. One election contains multiple candidates. **(1: N)**
6. One election produces multiple results. **(1: N)**
7. Each candidate produces one result per election. **(1: 1)**
8. One admin oversees multiple elections. **(1: N)**
9. One admin audit multiple result. **(1: M)**

Constraints

My system enforces the following constraints:

1. A **voter** can **vote** only once per **election**. This is enforced using a unique constraint on the combination of **voter_id** and **election_id** in the **Vote** entity.

2. Each **voter** must have a unique **citizenship number**.

3. A **voter** must be at least **18 years old** to be eligible to **vote**.

4. Only voters **registered for** an election are allowed to cast a **vote** in that **election**.

5. **Vote** confidentiality is maintained using a **vote_hash** to prevent tempering.

6. **Foreign key** constraints ensure referential integrity between related entities.

ISA (Specialisation) Explanation

My system uses specialisation (ISA) to represent different categories of **Voters** and **Candidates**.

The **Voter** entity is divided into:

1. **Local_Voter**
2. **Overseas_Voter**

This specialisation is **disjoint and total**, meaning each **voter** belongs to exactly one category.

Similarly, the **Candidate** entity is divided into:

1. **Party_Candidate**
2. **Independent_Candidate**

This allows the system to distinguish candidates based on their political affiliation and improved data organisation.

Relational Schema

The ER diagram is converted into the following relational schema.

```
VOTER (
  voter_id PRIMARY KEY,
  full_name,
  citizenship_no UNIQUE,
  DOB,
  contact_no,
  is_eligible
)

LOCAL_VOTER (
  voter_id PRIMARY KEY,
  district,
  ward_no,
  province,
  Local_body,
  FOREIGN KEY (voter_id) REFERENCES VOTER (voter_id)
)
```

```
OVERSEAS_VOTER (
  voter_id PRIMARY KEY,
  passport_no UNIQUE,
  embassy,
  country_resident,
  FOREIGN KEY (voter_id) REFERENCES VOTER (voter_id)
)

ELECTION (
  election_id PRIMARY KEY,
  election_name,
  election_date,
  status
)
```

```
CANDIDATE (
  candidate_id PRIMARY KEY,
  election_id,
  full_name,
  nominee_sig,
  DOB,
  FOREIGN KEY (election_id) REFERENCES ELECTION
(election_id)
)

PARTY_CANDIDATE (
  candidate_id PRIMARY KEY,
  party_name,
  party_symbol,
  party_ticket_no,
  FOREIGN KEY (candidate_id) REFERENCES CANDIDATE
(candidate_id)
)
```

```
INDEPENDENT_CANDIDATE (
  candidate_id PRIMARY KEY,
  endorsement,
  self_nom_ref,
  FOREIGN KEY (candidate_id) REFERENCES CANDIDATE
(candidate_id)
)

VOTE (
  vote_id PRIMARY KEY,
  voter_id,
  election_id,
  candidate_id,
  vote_time,
  vote_hash UNIQUE,
  FOREIGN KEY (voter_id) REFERENCES VOTER (voter_id),
  FOREIGN KEY (election_id) REFERENCES ELECTION
(election_id),
  FOREIGN KEY (candidate_id) REFERENCES CANDIDATE
(candidate_id),
  UNIQUE (voter_id, election_id)
)
```

```
RESULT (
  result_id PRIMARY KEY,
  election_id,
  candidate_id,
  admin_id,
  total_votes,
  last_updated,
  rank,
  FOREIGN KEY (election_id) REFERENCES ELECTION
(election_id),
  FOREIGN KEY (candidate_id) REFERENCES CANDIDATE
(candidate_id),
  FOREIGN KEY (admin_id) REFERENCES ADMIN
(admin_id)
)
ADMIN (
  admin_id PRIMARY KEY,
  role,
  pwd_hash,
  active_status,
  last_login,
  full_name
)
```

```
MANAGES (
  admin_id,
  election_id,
  PRIMARY KEY (admin_id, election_id),
  FOREIGN KEY (admin_id) REFERENCES ADMIN (admin_id),
  FOREIGN KEY (election_id) REFERENCES ELECTION (election_id)
)

REGISTERED_FOR (
  voter_id,
  election_id,
  PRIMARY KEY (voter_id, election_id),
  FOREIGN KEY (voter_id) REFERENCES VOTER (voter_id),
  FOREIGN KEY (election_id) REFERENCES ELECTION (election_id)
)
```

Conclusion

In conclusion, the **ER Diagram** successfully models an **Electronic Voting System** by adding up all the required entities, relationship, and constraints.

The inclusion of **voter registration** enhances the realism of my system, while constraints ensure data safety and integrity.

The **Relational Schema** implemented from the **ER Diagram** provides solid foundation for the system in real world DB Environment.

Tools and Technologies Used:

1. Draw.io

I used draw.io to bring my imagination of **Electronic Voting System** into visual representation using **ER Diagram** method.